

JavaScript Interview Handbook

Basic to Advanced — Answers, Examples & Interview Tips

Part A — Fundamentals & Core Concepts

1. JavaScript Basics

Q1. What is JavaScript?

Answer: JavaScript is a programming language used to add logic and interactivity to web pages. It also runs on servers through Node.js.

```
let message = 'Hello';
```

Tip: JavaScript and Java are different languages.

Q2. Where can JavaScript run?

Answer: It runs in web browsers, Node.js, mobile frameworks, desktop apps, and serverless platforms.

```
console.log('Runs in browser or Node.js');
```

Tip: Mention browser and Node.js in interviews.

Q3. Is JavaScript compiled or interpreted?

Answer: Modern engines use both. They parse and interpret code, then JIT-compile frequently used parts for speed.

```
V8 uses JIT compilation.
```

Tip: Avoid answering only 'interpreted'.

Q4. What is ECMAScript?

Answer: ECMAScript is the language standard. JavaScript is an implementation of that standard.

```
ES2015 introduced let, const, classes and modules.
```

Tip: ES6 and ES2015 mean the same edition.

Q5. What is strict mode?

Answer: Strict mode applies safer rules and catches silent errors.

```
'use strict';
```

Tip: Modules are strict by default.

Q6. How do you add JavaScript to HTML?

Answer: Use inline code, an internal script block, or an external script file.

```
<script src='app.js'></script>
```

Tip: External files are preferred.

Q7. What does defer do?

Answer: It downloads the script in parallel and executes it after HTML parsing, in document order.

```
<script src='app.js' defer></script>
```

Tip: Good default for page scripts.

Q8. What does async do?

Answer: It downloads in parallel and executes as soon as ready. Execution order is not guaranteed.

```
<script src='analytics.js' async></script>
```

Tip: Use for independent scripts.

Q9. defer vs async?

Answer: defer waits for HTML parsing and keeps order. async runs immediately when downloaded and may change order.

Use defer for app scripts; async for analytics.

Tip: Mention execution order.

Q10. What is a statement?

Answer: A statement performs an action, such as declaring a variable or running a condition.

```
let total = 10;
```

Tip: Statements often end with a semicolon.

Q11. What is an expression?

Answer: An expression produces a value.

```
2 + 3
```

Tip: Expressions can appear inside statements.

Q12. Are semicolons required?

Answer: Usually optional because of automatic semicolon insertion, but consistent semicolon use avoids edge cases.

```
const x = 10;
```

Tip: Do not rely blindly on ASI.

2. Variables and Declarations

Q13. What is a variable?

Answer: A variable is a named reference used to store a value.

```
let age = 25;
```

Tip: Use meaningful names.

Q14. var vs let vs const?

Answer: var is function-scoped and can be redeclared. let is block-scoped and reassignable. const is block-scoped and cannot be reassigned.

```
const name='Sam'; let count=0;
```

Tip: Prefer const, then let.

Q15. Can a const object be changed?

Answer: Yes. The binding cannot be reassigned, but object properties can be modified.

```
const user={name:'A'}; user.name='B';
```

Tip: const does not make objects immutable.

Q16. What is block scope?

Answer: A variable exists only inside the nearest block enclosed by braces.

```
if(true){ let x=1; }
```

Tip: let and const are block-scoped.

Q17. What is function scope?

Answer: A variable declared with var exists throughout the function.

```
function f(){ var x=1; }
```

Tip: var ignores ordinary block boundaries.

Q18. What is global scope?

Answer: A global variable is accessible from many parts of the program. Too many globals can cause conflicts.

```
const appName='Portal';
```

Tip: Avoid unnecessary globals.

Q19. Can let be redeclared?

Answer: Not in the same scope. It may be declared again in a different nested scope.

```
let x=1; { let x=2; }
```

Tip: Redeclaration differs from reassignment.

Q20. Can const be declared without a value?

Answer: No. const requires initialization at declaration.

```
const API_URL='/api';
```

Tip: let can be declared first and assigned later.

Q21. What is variable shadowing?

Answer: An inner variable uses the same name as an outer variable and temporarily hides it.

```
let x=1; { let x=2; }
```

Tip: Use carefully for readability.

Q22. What is illegal shadowing?

Answer: A block-scoped declaration cannot legally shadow certain var declarations in the same function structure.

```
let x=1; { /* var x=2 is invalid */ }
```

Tip: Know scope rules, not just names.

3. Data Types

Q23. What are JavaScript primitive types?

Answer: string, number, bigint, boolean, undefined, null, symbol.

```
let id=10n;
```

Tip: Primitive values are immutable.

Q24. What is a non-primitive type?

Answer: Objects are non-primitive. Arrays, functions, dates and maps are object-based values.

```
const user={name:'A'};
```

Tip: Functions are callable objects.

Q25. What is undefined?

Answer: It usually means a variable exists but has not been assigned a value.

```
let value; // undefined
```

Tip: Missing properties also return undefined.

Q26. What is null?

Answer: null is an intentional empty value assigned by the developer.

```
let selectedUser=null;
```

Tip: typeof null is 'object' due to a historic bug.

Q27. null vs undefined?

Answer: undefined usually means missing or unassigned. null usually means intentionally empty.

```
user.middleName ?? 'N/A'
```

Tip: Use strict equality when comparing.

Q28. What does typeof return?

Answer: It returns a string describing the operand's type.

```
typeof 10 // 'number'
```

Tip: typeof null returns 'object'.

Q29. How do you check an array?

Answer: Use Array.isArray(value).

```
Array.isArray([]) // true
```

Tip: typeof [] is 'object'.

Q30. What is NaN?

Answer: NaN means Not-a-Number and represents an invalid numeric result.

```
Number('abc') // NaN
```

Tip: NaN is still of type number.

Q31. How do you test NaN?

Answer: Use Number.isNaN(value) for a strict check.

```
Number.isNaN(NaN) // true
```

Tip: Global isNaN performs coercion.

Q32. Why is NaN not equal to itself?

Answer: NaN represents an undefined numeric result, so comparisons with it return false.

```
NaN === NaN // false
```

Tip: Use Number.isNaN.

Q33. What is BigInt?

Answer: BigInt stores integers larger than Number.MAX_SAFE_INTEGER.

```
const n=9007199254740993n;
```

Tip: Do not mix BigInt and Number directly.

Q34. What is Symbol?

Answer: Symbol creates unique identifiers, often used as object keys.

```
const key=Symbol('id');
```

Tip: Symbols avoid key collisions.

Q35. Primitive vs reference assignment?

Answer: Primitives are copied by value. Objects copy a reference to the same object.

```
const a={x:1}; const b=a;
```

Tip: Mutating b also affects a.

Q36. Is JavaScript dynamically typed?

Answer: Yes. Variable types are determined at runtime and may change.

```
let x=10; x='ten';
```

Tip: Dynamic typing differs from weak typing.

4. Operators and Equality

Q37. What are arithmetic operators?

Answer: They perform numeric operations: +, -, *, /, %, **, ++ and --.

```
2 ** 3 // 8
```

Tip: % gives the remainder.

Q38. What does + do with strings?

Answer: If either operand is a string, + may concatenate after coercion.

```
'5' + 2 // '52'
```

Tip: Other arithmetic operators usually convert to numbers.

Q39. == vs ===?

Answer: == allows type coercion. === compares both type and value.

```
0 == false // true; 0 === false // false
```

Tip: Prefer strict equality.

Q40. != vs !==?

Answer: != allows coercion. !== compares type and value strictly.

```
'5' != 5 // false
```

Tip: Prefer !==.

Q41. What is the ternary operator?

Answer: A compact conditional expression that returns one of two values.

```
const label = age >= 18 ? 'Adult' : 'Minor';
```

Tip: Avoid deeply nested ternaries.

Q42. What are logical operators?

Answer: &&, || and ! combine or invert conditions. They also use short-circuit evaluation.

```
isReady && start();
```

Tip: They return operands, not always booleans.

Q43. What does && return?

Answer: It returns the first falsy operand, or the last operand if all are truthy.

```
0 && 'x' // 0
```

Tip: Useful for conditional execution.

Q44. What does || return?

Answer: It returns the first truthy operand, or the last operand.

```
name || 'Guest'
```

Tip: It treats 0 and empty string as missing.

Q45. What does ?? return?

Answer: Nullish coalescing returns the right value only when the left is null or undefined.

```
count ?? 0
```

Tip: Use it when 0 and "" are valid.

Q46. || vs ??

Answer: || falls back for any falsy value. ?? falls back only for null or undefined.

```
0 || 10 // 10; 0 ?? 10 // 0
```

Tip: Common interview question.

Q47. What is optional chaining?

Answer: It safely accesses a property or method and returns undefined if an earlier value is nullish.

```
user.address?.city
```

Tip: Combine with ?? for defaults.

Q48. What are bitwise operators?

Answer: They work on 32-bit integer representations: &, |, ^, ~, <<, >> and >>>.

```
5 & 1 // 1
```

Tip: Rare in typical web code.

Q49. What is operator precedence?

Answer: It determines which operators are evaluated first.

```
2 + 3 * 4 // 14
```

Tip: Use parentheses for clarity.

Q50. What is short-circuit evaluation?

Answer: Logical evaluation stops as soon as the final result is known.

```
isAdmin || showLogin();
```

Tip: Can be used for defaults and guards.

5. Type Conversion and Truthiness

Q51. What is implicit coercion?

Answer: JavaScript automatically converts a value during an operation or comparison.

```
'5' * 2 // 10
```

Tip: Implicit coercion can surprise developers.

Q52. What is explicit conversion?

Answer: The developer intentionally converts a value using String, Number or Boolean.

```
Number('42') // 42
```

Tip: Preferred for clarity.

Q53. How do you convert to string?

Answer: Use String(value), template literals, or toString when the value is not nullish.

```
String(100)
```

Tip: String(null) is safe.

Q54. How do you convert to number?

Answer: Use Number, parseInt, parseFloat or unary plus depending on the input.

```
Number('12.5')
```

Tip: Number("") returns 0.

Q55. parseInt vs Number?

Answer: parseInt reads an integer prefix. Number requires the whole string to be a valid number.

```
parseInt('10px') // 10
```

Tip: Pass radix 10 when appropriate.

Q56. What are falsy values?

Answer: false, 0, -0, 0n, empty string, null, undefined and NaN.

```
Boolean('') // false
```

Tip: Empty arrays and objects are truthy.

Q57. What are truthy values?

Answer: All values not in the falsy list are truthy.

```
Boolean([]) // true
```

Tip: The string 'false' is truthy.

Q58. How do you convert to boolean?

Answer: Use Boolean(value) or double negation.

```
!!'hello' // true
```

Tip: Boolean is clearer in production code.

Q59. What is numeric precision issue?

Answer: JavaScript uses binary floating-point, so some decimal calculations are not exact.

```
0.1 + 0.2 !== 0.3
```

Tip: Use rounding or decimal libraries for money.

6. Scope, Hoisting and Execution

Q60. What is lexical scope?

Answer: Scope is determined by where code is written, not where a function is called.

```
function outer(){ let x=1; function inner(){return x;} }
```

Tip: Closures depend on lexical scope.

Q61. What is the scope chain?

Answer: When a variable is not found locally, JavaScript searches outer lexical scopes.

```
inner function reads outer variable
```

Tip: The search ends at global scope.

Q62. What is hoisting?

Answer: Declarations are processed before execution. Access behavior depends on var, let, const and function type.

```
console.log(x); var x=1;
```

Tip: Declarations are not physically moved.

Q63. How is var hoisted?

Answer: The declaration is initialized with undefined before execution.

```
console.log(x); var x=1; // undefined
```

Tip: Assignment stays in place.

Q64. How are let and const hoisted?

Answer: Their bindings exist but cannot be accessed before declaration because of the temporal dead zone.

```
console.log(x); let x=1; // ReferenceError
```

Tip: They are hoisted but uninitialized.

Q65. What is the temporal dead zone?

Answer: The period from scope start until a let or const declaration is evaluated.

```
{ /* TDZ */ let x=1; }
```

Tip: Access during TDZ throws ReferenceError.

Q66. Are function declarations hoisted?

Answer: Yes. The complete function declaration is available before its written position.

```
say(); function say(){} 
```

Tip: Function expressions behave differently.

Q67. Are function expressions hoisted?

Answer: Only their variable declaration is hoisted according to var, let or const rules.

```
run(); const run=()=>{}; // error
```

Tip: Do not confuse declaration and expression.

Q68. What is an execution context?

Answer: It is the environment used to execute code, containing variables, scope links and this binding.

Global and function contexts are common.

Tip: Contexts are managed by the call stack.

Q69. What is the call stack?

Answer: A LIFO stack that tracks active function calls.

```
a() calls b(), b() sits above a()
```

Tip: Stack overflow occurs with excessive recursion.

7. Functions

Q70. What is a function declaration?

Answer: A named function defined with the function keyword and hoisted with its body.

```
function add(a,b){return a+b;}
```

Tip: Good for reusable named logic.

Q71. What is a function expression?

Answer: A function stored in a variable or property.

```
const add=function(a,b){return a+b;};
```

Tip: Its variable follows normal hoisting rules.

Q72. What is an arrow function?

Answer: A concise function syntax with lexical this and no own arguments object.

```
const add=(a,b)=>a+b;
```

Tip: Do not use as object methods when dynamic this is needed.

Q73. Arrow function vs regular function?

Answer: Arrow functions have lexical this, no own arguments, and cannot be used as constructors. Regular functions have dynamic this.

```
const f={()=>this;
```

Tip: Choose based on this behavior.

Q74. What are parameters and arguments?

Answer: Parameters are names in the function definition. Arguments are actual values passed during a call.

```
function greet(name){}; greet('Sam');
```

Tip: Extra arguments are allowed.

Q75. What are default parameters?

Answer: They provide a value when an argument is missing or undefined.

```
function greet(name='Guest'){}
```

Tip: Passing null does not use the default.

Q76. What is the arguments object?

Answer: An array-like object available in regular functions containing passed arguments.

```
function f(){ return arguments.length; }
```

Tip: Arrow functions do not have it.

Q77. What is a rest parameter?

Answer: It collects remaining arguments into a real array.

```
function sum(...nums){return nums.reduce((a,b)=>a+b,0);}
```

Tip: Rest must be the last parameter.

Q78. What is a callback?

Answer: A function passed to another function to be called later.

```
items.map(item=>item.name)
```

Tip: Callbacks may be synchronous or asynchronous.

Q79. What is a higher-order function?

Answer: A function that accepts another function, returns one, or both.

```
const twice=fn=>x=>fn(fn(x));
```

Tip: map, filter and reduce are examples.

Q80. What is a pure function?

Answer: It returns the same output for the same input and has no side effects.

```
const add=(a,b)=>a+b;
```

Tip: Pure functions are easier to test.

Q81. What is a side effect?

Answer: A change outside the function, such as modifying global state, DOM, files or network data.

```
document.title='Home';
```

Tip: Side effects are not always bad, but isolate them.

Q82. What is recursion?

Answer: A function calls itself until a base condition is reached.

```
function fact(n){return n<=1?1:n*fact(n-1);}
```

Tip: Always define a base case.

Q83. What is an IIFE?

Answer: An immediately invoked function expression runs as soon as it is created.

```
(function(){ console.log('run'); })();
```

Tip: Historically used for private scope.

Q84. What is function currying?

Answer: Currying converts a function with many arguments into a sequence of single-argument functions.

```
const add=a=>b=>a+b;
```

Tip: Useful for reusable partial configuration.

Q85. What is partial application?

Answer: It pre-fills some arguments and returns a new function expecting the rest.

```
const add10 = x => add(10,x);
```

Tip: Related to, but not identical to, currying.

8. Strings

Q86. How are strings created?

Answer: Use single quotes, double quotes or template literals.

```
const name=`Sam`;
```

Tip: Template literals support interpolation.

Q87. Are strings mutable?

Answer: No. String methods return new strings instead of changing the original.

```
let s='abc'; s[0]='x'; // unchanged
```

Tip: Reassign to store the result.

Q88. How do template literals work?

Answer: Backticks allow interpolation and multiline strings.

```
`Hello ${name}`
```

Tip: Use `${expression}`.

Q89. How do you get string length?

Answer: Use the length property.

```
'hello'.length // 5
```

Tip: length counts UTF-16 code units.

Q90. slice vs substring?

Answer: slice supports negative indexes. substring converts negative values to zero and may swap arguments.

```
'hello'.slice(-2) // 'lo'
```

Tip: slice is usually clearer.

Q91. How do you search a string?

Answer: Use includes, indexOf, startsWith or endsWith.

```
'javascript'.includes('script')
```

Tip: includes returns a boolean.

Q92. How do you replace text?

Answer: Use `replace` for the first match or `replaceAll` for all string matches.

```
'a-a'.replaceAll('a','b')
```

Tip: Regex can also replace globally.

Q93. How do you split a string?

Answer: `split` returns an array using a separator.

```
'a,b,c'.split(',')
```

Tip: Use `join` to combine arrays.

Q94. How do you remove spaces?

Answer: `trim` removes both ends; `trimStart` and `trimEnd` remove one side.

```
' hi '.trim()
```

Tip: It does not remove inner spaces.

Q95. How do you change case?

Answer: Use `toUpperCase` and `toLowerCase`.

```
'Hi'.toLowerCase()
```

Tip: Locale-aware methods also exist.

9. Arrays

Q96. What is an array?

Answer: An ordered, zero-indexed collection that can hold mixed values.

```
const items=[1,'two',true];
```

Tip: Use `Array.isArray` to detect arrays.

Q97. How do you add elements?

Answer: `push` adds at the end; `unshift` adds at the beginning.

```
arr.push(4);
```

Tip: `push` is usually faster than `unshift`.

Q98. How do you remove elements?

Answer: `pop` removes from the end; `shift` removes from the beginning.

```
const last=arr.pop();
```

Tip: Both mutate the array.

Q99. slice vs splice?

Answer: `slice` returns a shallow copy without mutation. `splice` adds, removes or replaces items and mutates the array.

```
arr.slice(1,3); arr.splice(1,1);
```

Tip: Very common interview question.

Q100. What does map do?

Answer: `map` creates a new array by transforming every item.

```
users.map(u=>u.name)
```

Tip: Use `map` when you need the returned array.

Q101. What does filter do?

Answer: filter returns items whose callback result is truthy.

```
nums.filter(n=>n>0)
```

Tip: It does not change the original array.

Q102. What does reduce do?

Answer: reduce combines array items into one result using an accumulator.

```
nums.reduce((sum,n)=>sum+n,0)
```

Tip: Provide an initial value.

Q103. What does forEach do?

Answer: forEach runs a callback for each item and returns undefined.

```
items.forEach(console.log)
```

Tip: Do not expect a new array.

Q104. find vs filter?

Answer: find returns the first matching item. filter returns all matching items.

```
users.find(u=>u.id===1)
```

Tip: find may return undefined.

Q105. some vs every?

Answer: some checks whether at least one item matches. every requires all items to match.

```
nums.some(n=>n<0)
```

Tip: Both short-circuit.

Q106. includes vs indexOf?

Answer: includes returns a boolean and correctly finds NaN. indexOf returns an index or -1.

```
[NaN].includes(NaN) // true
```

Tip: Use includes for membership checks.

Q107. How do you copy an array?

Answer: Use spread, slice or Array.from for a shallow copy.

```
const copy=[...arr];
```

Tip: Nested objects are still shared.

Q108. How do you merge arrays?

Answer: Use spread or concat.

```
const all=[...a,...b];
```

Tip: Both create a new array.

Q109. How do you remove duplicates?

Answer: Convert to a Set and back to an array.

```
const unique=[...new Set(arr)];
```

Tip: Works well for primitive values.

Q110. How does sort work by default?

Answer: It sorts values as strings and mutates the array.

```
[10,2].sort() // [10,2]
```

Tip: Use (a,b)=>a-b for numbers.

Q111. How do you flatten an array?

Answer: Use flat with a depth value.

```
[1, [2, [3]]].flat(2)
```

Tip: flat(Infinity) handles unknown depth.

Q112. What is array destructuring?

Answer: It extracts array items into variables by position.

```
const [first, second]=arr;
```

Tip: Use commas to skip items.

Q113. What is sparse array?

Answer: An array with empty slots rather than explicit undefined values.

```
const a=[]; a[3]='x';
```

Tip: Some methods skip empty slots.

10. Objects

Q114. What is an object?

Answer: A collection of key-value properties.

```
const user={name:'Sam',age:25};
```

Tip: Keys are strings or symbols.

Q115. Dot vs bracket notation?

Answer: Dot notation uses fixed valid identifiers. Brackets support dynamic keys and special characters.

```
user.name; user[key];
```

Tip: Use brackets for computed property names.

Q116. How do you add or update a property?

Answer: Assign a value using dot or bracket notation.

```
user.city='Chennai';
```

Tip: Assignment creates the property if missing.

Q117. How do you delete a property?

Answer: Use the delete operator.

```
delete user.age;
```

Tip: Prefer creating a new object in immutable flows.

Q118. How do you check if a key exists?

Answer: Use Object.hasOwn for own properties or in for own and inherited properties.

```
Object.hasOwn(user, 'name')
```

Tip: hasOwn is safer than direct method calls.

Q119. What does Object.keys return?

Answer: An array of own enumerable string keys.

```
Object.keys(user)
```

Tip: Symbols are not included.

Q120. What does `Object.values` return?

Answer: An array of own enumerable values.

```
Object.values(user)
```

Tip: Useful for iteration.

Q121. What does `Object.entries` return?

Answer: An array of [key, value] pairs.

```
Object.entries(user)
```

Tip: Combine with `Object.fromEntries`.

Q122. What is object destructuring?

Answer: It extracts named properties into variables.

```
const {name, age}=user;
```

Tip: Aliases and defaults are supported.

Q123. What is a computed property?

Answer: A property name evaluated from an expression inside brackets.

```
const key='id'; const obj={{[key]:1};
```

Tip: Useful for dynamic objects.

Q124. What is method shorthand?

Answer: A concise way to define object methods.

```
const obj={ greet(){return 'Hi';} };
```

Tip: Its this is dynamic like a regular function.

Q125. What is shallow copy of object?

Answer: A new top-level object whose nested references are still shared.

```
const copy={...user};
```

Tip: Spread is not a deep clone.

Q126. How do you merge objects?

Answer: Use object spread or `Object.assign`. Later properties overwrite earlier ones.

```
const merged={...a,...b};
```

Tip: Both are shallow.

Q127. What does `Object.freeze` do?

Answer: It prevents adding, deleting or changing top-level properties.

```
Object.freeze(config);
```

Tip: It is shallow.

Q128. What does `Object.seal` do?

Answer: It prevents adding or deleting properties, but existing writable properties may change.

```
Object.seal(obj);
```

Tip: Seal differs from freeze.

Q129. What is JSON?

Answer: A text format for data exchange supporting objects, arrays, strings, numbers, booleans and null.

```
JSON.stringify(user)
```

Tip: Functions and undefined are not standard JSON values.

Q130. JSON.stringify vs JSON.parse?

Answer: stringify converts a JavaScript value to JSON text. parse converts JSON text into a JavaScript value.

```
JSON.parse('{"x":1}')
```

Tip: Parsing invalid JSON throws.

11. Conditionals, Loops and Errors

Q131. How does if...else work?

Answer: It runs different blocks based on truthy or falsy conditions.

```
if(score>=50){pass();}else{fail();}
```

Tip: Keep conditions readable.

Q132. When should switch be used?

Answer: Use switch when comparing one expression against many fixed values.

```
switch(role){case 'admin': ...}
```

Tip: Remember break unless fall-through is intended.

Q133. for vs while?

Answer: for is convenient when initialization, condition and update are known. while is useful when repetition count is uncertain.

```
for(let i=0;i<3;i++){}
```

Tip: Either can express the same logic.

Q134. for...of vs for...in?

Answer: for...of iterates iterable values. for...in iterates enumerable property keys.

```
for(const value of arr){}
```

Tip: Avoid for...in for ordinary arrays.

Q135. What does break do?

Answer: It exits the nearest loop or switch.

```
if(found) break;
```

Tip: Use labels only in rare nested-loop cases.

Q136. What does continue do?

Answer: It skips the current loop iteration and moves to the next one.

```
if(!item) continue;
```

Tip: Do not overuse it.

Q137. What is a try...catch block?

Answer: It handles exceptions thrown while executing the try block.

```
try{ risky(); }catch(err){ log(err); }
```

Tip: Only catches synchronous errors in that call path.

Q138. What does finally do?

Answer: It runs after try/catch whether an error occurred or not.

```
finally{ closeConnection(); }
```

Tip: Useful for cleanup.

Q139. How do you throw an error?

Answer: Use throw with an Error object.

```
throw new Error('Invalid input');
```

Tip: Throw Error objects for stack traces.

Q140. Error vs custom error?

Answer: Built-in Error is general. Custom classes add domain-specific names or fields.

```
class ValidationError extends Error {}
```

Tip: Call super(message).

Q141. What is defensive programming?

Answer: It validates assumptions and handles unexpected input before failures spread.

```
if(!Array.isArray(items)) throw new TypeError();
```

Tip: Validate at system boundaries.

Q142. What is debugging?

Answer: The process of finding and fixing defects using logs, breakpoints, stack traces and tests.

```
console.table(users);
```

Tip: Use browser DevTools breakpoints. Congratulations! You have completed Continue with Advanced JavaScript, Async, DOM & Coding. Revise the examples, explain each answer aloud, and practise coding without copying.

Part B — Advanced JavaScript, Async, DOM & Coding

12. ES6+ Features

Q143. What did ES6 introduce?

Answer: Major additions include let, const, arrow functions, classes, modules, promises, template literals, destructuring, spread, rest, Map and Set.

```
const add=(a,b)=>a+b;
```

Tip: ES6 is also called ES2015.

Q144. What is destructuring?

Answer: A concise way to extract values from arrays or properties from objects.

```
const {name}=user; const [first]=items;
```

Tip: Defaults and aliases are supported.

Q145. What is object rest syntax?

Answer: It collects remaining properties into a new object.

```
const {id,...rest}=user;
```

Tip: It creates a shallow copy.

Q146. What is spread syntax?

Answer: It expands iterable values or object properties into another structure.

```
const copy=[...items]; const next={...user,active:true};
```

Tip: Spread is shallow.

Q147. What are default parameters?

Answer: They supply a value when an argument is omitted or undefined.

```
function greet(name='Guest'){};
```

Tip: null does not trigger the default.

Q148. What is a Set?

Answer: A collection of unique values.

```
const unique=new Set([1,1,2]);
```

Tip: Useful for duplicate removal.

Q149. What is a Map?

Answer: A key-value collection where keys may be any type and insertion order is preserved.

```
const map=new Map([[user, 'admin']]);
```

Tip: Better than objects for dynamic keys.

Q150. Map vs Object?

Answer: Map supports any key type, direct size, predictable iteration and built-in methods. Objects suit records with known string keys.

```
map.set(key, value);
```

Tip: Choose based on data meaning.

Q151. Set vs Array?

Answer: Set enforces uniqueness and offers fast membership checks. Array preserves duplicates and supports indexed operations.

```
set.has(value)
```

Tip: Convert when array methods are needed.

Q152. What are WeakMap and WeakSet?

Answer: They hold weak references to object keys or values, allowing garbage collection when no other references exist.

```
const cache=new WeakMap();
```

Tip: They are not enumerable.

Q153. What is a JavaScript module?

Answer: A file with its own scope that exports and imports values.

```
export const api={}; import {api} from './api.js';
```

Tip: Modules are strict by default.

Q154. Named vs default export?

Answer: A module may have many named exports but only one default export.

```
export default App; export const helper={()=>{}};
```

Tip: Named imports must use exported names unless aliased.

Q155. Dynamic import?

Answer: import() loads a module asynchronously and returns a promise.

```
const mod=await import('./chart.js');
```

Tip: Useful for code splitting.

Q156. What is optional catch binding?

Answer: A catch block may omit the error variable when it is unused.

```
try{}catch{}
```

Tip: Keeps code concise.

Q157. What is logical assignment?

Answer: Operators such as ||=, &&= and ??= combine a logical test with assignment.

```
config.timeout ??= 5000;
```

Tip: Use ??= when zero is valid.

13. Closures and Functional Concepts

Q158. What is a closure?

Answer: A function retains access to variables from its lexical scope even after the outer function finishes.

```
function counter(){let n=0; return()=>++n;}
```

Tip: Used for private state and callbacks.

Q159. Where are closures used?

Answer: Event handlers, factories, memoization, currying, modules and private state.

```
const next=counter();
```

Tip: Give a practical project example.

Q160. Can closures cause memory issues?

Answer: Yes. Referenced outer data remains reachable while the closure exists.

```
handler closes over a large object
```

Tip: Remove unused listeners and references.

Q161. What is a function factory?

Answer: A function that creates and returns configured functions.

```
const multiply=a=>b=>a*b;
```

Tip: Closures preserve configuration.

Q162. What is memoization?

Answer: Caching a function result based on its inputs to avoid repeated work.

```
cache[key] ??= expensive(key);
```

Tip: Best for pure functions.

Q163. What is composition?

Answer: Combining small functions so one function's output becomes another's input.

```
const compose=(f,g)=>x=>f(g(x));
```

Tip: Encourages reusable logic.

Q164. What is immutability?

Answer: Updating data by creating new values instead of changing existing ones.

```
const next={...user, name: 'New'};
```

Tip: Useful for predictable state updates.

Q165. What is referential transparency?

Answer: An expression can be replaced by its value without changing behavior.

`add(2,3)` can be replaced by `5`

Tip: A property of pure code.

Q166. What is first-class function?

Answer: Functions can be stored, passed and returned like other values.

`const fn=console.log;`

Tip: Foundation of functional patterns.

Q167. What is currying used for?

Answer: It creates reusable specialized functions and supports composition.

`const withTax=rate=>amount=>amount*(1+rate);`

Tip: Explain with a simple use case.

14. this, call, apply, bind

Q168. What is this?

Answer: this is a runtime binding determined mainly by how a regular function is called.

`obj.show();`

Tip: Arrow functions capture this lexically.

Q169. What is implicit binding?

Answer: When a method is called through an object, this usually refers to that object.

`user.greet();`

Tip: Losing the method reference loses the binding.

Q170. What is explicit binding?

Answer: call, apply and bind manually set this.

`fn.call(user,1,2);`

Tip: call and apply invoke immediately.

Q171. call vs apply?

Answer: Both invoke immediately with a chosen this. call takes separate arguments; apply takes an array-like list.

`fn.call(obj,a,b); fn.apply(obj,[a,b]);`

Tip: Modern spread often replaces apply.

Q172. What does bind do?

Answer: bind returns a new function with fixed this and optional preset arguments.

`const bound=fn.bind(obj);`

Tip: It does not invoke immediately.

Q173. What is new binding?

Answer: When called with new, a function's this refers to the newly created object.

`const p=new Person();`

Tip: Constructors should not be arrow functions.

Q174. What is lexical this?

Answer: Arrow functions use this from the surrounding scope rather than creating their own.

```
setTimeout(()=>this.save(),0);
```

Tip: Useful inside class methods.

Q175. Why does a detached method lose this?

Answer: Calling the function without its owner changes the call-site binding.

```
const f=user.greet; f();
```

Tip: Use bind or a wrapper.

Q176. What is method borrowing?

Answer: Using call or apply to run one object's method with another object.

```
Array.prototype.join.call(arrayLike, ',')
```

Tip: Use carefully for readability.

15. Prototypes and Classes

Q177. What is a prototype?

Answer: An object used as a fallback source of properties for another object.

```
user.__proto__ === User.prototype
```

Tip: Prefer Object.getPrototypeOf over __proto__.

Q178. What is the prototype chain?

Answer: A sequence of linked prototypes searched when a property is not found directly.

```
obj -> prototype -> Object.prototype -> null
```

Tip: Property lookup walks the chain.

Q179. How does inheritance work in JavaScript?

Answer: Objects inherit through prototype links. Class syntax is a clearer layer over this system.

```
class Admin extends User {}
```

Tip: Classes still use prototypes.

Q180. What is a constructor function?

Answer: A regular function intended to create objects with new.

```
function User(name){this.name=name;}
```

Tip: Methods should usually go on the prototype.

Q181. What does new do?

Answer: It creates an object, links its prototype, binds this, runs the constructor and returns the object unless another object is returned.

```
new User('Sam')
```

Tip: Know the four main steps.

Q182. What is a class?

Answer: Syntactic structure for constructors, prototype methods, inheritance and static members.

```
class User{ constructor(name){this.name=name;} }
```

Tip: Class declarations are in TDZ.

Q183. Instance vs static method?

Answer: Instance methods are called on objects. Static methods are called on the class itself.

```
User.create(); user.save();
```

Tip: Static methods are not inherited by instances.

Q184. What does super do?

Answer: It calls a parent constructor or accesses parent methods.

```
super(name); super.save();
```

Tip: Call super before using this in derived constructors.

Q185. What are private class fields?

Answer: Fields beginning with # are accessible only inside the class body.

```
class A{ #value=1; }
```

Tip: They are enforced by the language.

Q186. What are getters and setters?

Answer: Special methods that read or write a property-like interface.

```
get fullName(){return this.first+' '+this.last;}
```

Tip: Avoid hidden expensive work.

Q187. Object.create use?

Answer: It creates an object with a chosen prototype.

```
const child=Object.create(parent);
```

Tip: Useful for direct prototype control.

Q188. hasOwn vs inherited property?

Answer: Object.hasOwn checks only direct properties; the in operator also checks prototypes.

```
Object.hasOwn(obj, 'x')
```

Tip: Important for safe iteration.

16. Asynchronous JavaScript and Event Loop

Q189. What is synchronous code?

Answer: It runs one operation at a time in order on the current call stack.

```
const x=calculate();
```

Tip: Long tasks block the main thread.

Q190. What is asynchronous code?

Answer: It starts work that completes later, allowing other code to continue.

```
fetch('/api').then(...)
```

Tip: JavaScript coordinates async work through host APIs and queues.

Q191. What is the event loop?

Answer: It moves eligible queued callbacks to the call stack when the stack is empty.

```
setTimeout(()=>{},0)
```

Tip: Explain stack, host APIs and queues.

Q192. What is a macrotask?

Answer: A scheduled task such as timers, many I/O callbacks and UI events.

```
setTimeout(callback, 0)
```

Tip: Usually one task runs per event-loop turn.

Q193. What is a microtask?

Answer: A high-priority queued job such as promise reactions and queueMicrotask callbacks.

```
Promise.resolve().then(fn)
```

Tip: Microtasks run before the next macrotask.

Q194. Promise callback vs setTimeout order?

Answer: Promise callbacks normally run before zero-delay timer callbacks because microtasks are drained first.

```
Promise.resolve().then(()=>1); setTimeout(()=>2);
```

Tip: Common output question.

Q195. What is callback hell?

Answer: Deeply nested callbacks that make flow and error handling hard to maintain.

```
load(a=>save(b=>notify(...)))
```

Tip: Promises and async/await improve structure.

Q196. What is a Promise?

Answer: An object representing a future result with pending, fulfilled or rejected states.

```
const p=fetch('/api');
```

Tip: A promise settles only once.

Q197. What does then return?

Answer: then returns a new promise, enabling chaining.

```
fetch(url).then(r=>r.json()).then(data=>...)
```

Tip: Return values or promises from callbacks.

Q198. What does catch do?

Answer: It handles a rejection from an earlier point in the promise chain.

```
task().catch(handleError)
```

Tip: It also returns a promise.

Q199. What does finally do in promises?

Answer: It runs after settlement and does not receive the result directly.

```
task().finally(stopLoader)
```

Tip: Useful for cleanup.

Q200. What is async function?

Answer: A function that always returns a promise and allows await inside.

```
async function load(){ return 1; }
```

Tip: Returning 1 becomes Promise.resolve(1).

Q201. What does await do?

Answer: It pauses only the async function until a promise settles, then resumes through a microtask.

```
const data=await fetch(url);
```

Tip: It does not block the entire thread.

Q202. How do you handle await errors?

Answer: Use try/catch or attach catch to the returned promise.

```
try{await save();}catch(e){}
```

Tip: Handle errors at an appropriate layer.

Q203. What is Promise.all?

Answer: It fulfills when all inputs fulfill and rejects on the first rejection.

```
await Promise.all([a(),b()])
```

Tip: Use for independent parallel work.

Q204. What is Promise.allSettled?

Answer: It waits for every input and returns each fulfillment or rejection result.

```
await Promise.allSettled(tasks)
```

Tip: Useful when partial failure is acceptable.

Q205. What is Promise.race?

Answer: It settles with the first input to settle.

```
Promise.race([request,timeout])
```

Tip: The other operations continue unless cancelled.

Q206. What is Promise.any?

Answer: It fulfills with the first fulfilled input and rejects only if all reject.

```
Promise.any(mirrors)
```

Tip: Rejection is an AggregateError.

Q207. Sequential vs parallel await?

Answer: Sequential await waits one-by-one. Starting promises first and awaiting together allows parallel execution.

```
const [a,b]=await Promise.all([getA(),getB()]);
```

Tip: Use parallel only for independent work.

Q208. What is queueMicrotask?

Answer: It schedules a callback in the microtask queue.

```
queueMicrotask(updateState);
```

Tip: Use carefully to avoid starving tasks.

17. DOM and Events

Q209. What is the DOM?

Answer: A tree-like object representation of an HTML document that JavaScript can read and modify.

```
document.querySelector('#app')
```

Tip: DOM APIs are provided by the browser.

Q210. How do you select elements?

Answer: Use querySelector, querySelectorAll, getElementById and related methods.

```
document.querySelector('.btn')
```

Tip: querySelector returns the first match.

Q211. NodeList vs HTMLCollection?

Answer: NodeList may be static or live depending on the API. HTMLCollection is usually live and contains elements.

```
document.querySelectorAll('li')
```

Tip: Convert with Array.from when array methods are needed.

Q212. How do you create and add an element?

Answer: Use createElement, set properties, then append or prepend.

```
const li=document.createElement('li'); list.append(li);
```

Tip: Prefer textContent for plain text.

Q213. textContent vs innerHTML?

Answer: textContent treats content as text. innerHTML parses HTML and can create security risks with untrusted input.

```
el.textContent=userInput;
```

Tip: Avoid unsafe HTML injection.

Q214. How do you remove an element?

Answer: Call remove on the element or removeChild on its parent.

```
element.remove();
```

Tip: Clean related listeners when needed.

Q215. What is event bubbling?

Answer: An event travels from the target upward through ancestors after the target phase.

button click bubbles to container

Tip: Most common event behavior.

Q216. What is event capturing?

Answer: An event travels from outer ancestors toward the target before bubbling.

```
addEventListener('click',fn,{capture:true})
```

Tip: Used less often.

Q217. What is event delegation?

Answer: Attach one listener to a parent and handle matching descendants through event.target.

```
list.addEventListener('click', e=>{...})
```

Tip: Good for dynamic lists.

Q218. target vs currentTarget?

Answer: target is where the event started. currentTarget is the element whose listener is running.

```
e.target; e.currentTarget;
```

Tip: Important in delegation.

Q219. preventDefault vs stopPropagation?

Answer: preventDefault cancels the browser action. stopPropagation stops event travel.

```
e.preventDefault();
```

Tip: They solve different problems.

Q220. What does once option do?

Answer: It removes the listener automatically after its first call.

```
el.addEventListener('click',fn,{once:true});
```

Tip: Useful for one-time actions.

Q221. How do you handle form submission?

Answer: Listen for submit, validate values and optionally prevent the default navigation.

```
form.addEventListener('submit',e=>e.preventDefault());
```

Tip: Prefer submit over button click.

Q222. What is DOMContentLoaded?

Answer: It fires after HTML is parsed, without waiting for all images and other resources.

```
document.addEventListener('DOMContentLoaded',init);
```

Tip: defer often removes the need for it.

Q223. What is reflow?

Answer: The browser recalculates layout after geometry-affecting changes.

reading offsetHeight after writes may force layout

Tip: Batch reads and writes.

Q224. What is repaint?

Answer: The browser redraws visual pixels without necessarily recalculating layout.

changing color may repaint

Tip: Transforms and opacity are often cheaper.

18. Browser APIs, Storage and Networking

Q225. localStorage vs sessionStorage?

Answer: localStorage persists by origin until removed. sessionStorage lasts for the current tab session.

```
localStorage.setItem('theme','dark')
```

Tip: Both store strings.

Q226. How do you store objects in web storage?

Answer: Convert to JSON when saving and parse when reading.

```
localStorage.setItem('user',JSON.stringify(user));
```

Tip: Handle invalid or missing data.

Q227. What are cookies?

Answer: Small key-value data sent with matching HTTP requests, subject to domain, path, expiry and security attributes.

```
document.cookie='theme=dark';
```

Tip: Sensitive auth cookies should be HttpOnly and Secure.

Q228. Cookies vs localStorage?

Answer: Cookies can be sent automatically with requests and support server-controlled security flags. localStorage is browser-only and larger but vulnerable to XSS access.

Use secure cookies for many session tokens.

Tip: Discuss security, not only size.

Q229. What is Fetch API?

Answer: A promise-based browser API for HTTP requests.

```
const res=await fetch('/api/users');
```

Tip: fetch rejects on network errors, not ordinary HTTP errors.

Q230. How do you handle HTTP errors with fetch?

Answer: Check response.ok or status and throw an error when needed.

```
if(!res.ok) throw new Error(`HTTP ${res.status}`);
```

Tip: Always check status.

Q231. How do you send JSON with fetch?

Answer: Set method, JSON content type and a stringified body.

```
fetch('/api',{method:'POST',headers:{'Content-Type':'application/json'},body:JSON.stringify(data)})
```

Tip: Do not stringify FormData.

Q232. What is AbortController?

Answer: It provides a signal to cancel supported operations such as fetch.

```
controller.abort();
```

Tip: Useful for navigation and search requests.

Q233. What is CORS?

Answer: A browser security mechanism controlling cross-origin access through server response headers.

```
Access-Control-Allow-Origin
```

Tip: CORS is configured mainly on the server.

Q234. What is same-origin policy?

Answer: Browsers restrict scripts from freely reading resources from a different scheme, host or port.

```
https://a.com vs https://api.a.com are different origins
```

Tip: CORS can permit controlled access.

Q235. What is JSONP?

Answer: An old script-tag technique for cross-origin GET requests.

```
callback({data:1})
```

Tip: Not used for modern APIs.

Q236. What is WebSocket?

Answer: A persistent full-duplex connection between client and server.

```
const ws=new WebSocket(url);
```

Tip: Good for chat and live updates.

Q237. What is Web Worker?

Answer: A background thread for CPU-heavy JavaScript without direct DOM access.

```
new Worker('worker.js')
```

Tip: Communicate through messages.

Q238. What is Service Worker?

Answer: A background browser worker that can intercept network requests and support caching, offline behavior and push.

```
navigator.serviceWorker.register('/sw.js')
```

Tip: Requires secure context except localhost.

19. Performance, Memory and Security

Q239. What is garbage collection?

Answer: Automatic removal of objects that are no longer reachable.

unreachable objects become eligible

Tip: Developers manage references, not manual free calls.

Q240. What causes memory leaks?

Answer: Long-lived references such as forgotten timers, listeners, caches, closures or detached DOM nodes.

window.addEventListener without cleanup

Tip: Use profiling tools.

Q241. What is debouncing?

Answer: It delays execution until calls stop for a set period.

debounce(search, 300)

Tip: Useful for search input.

Q242. What is throttling?

Answer: It limits execution to at most once per interval.

throttle(scrollHandler, 100)

Tip: Useful for scroll and resize.

Q243. Debounce vs throttle?

Answer: Debounce waits for inactivity. Throttle allows periodic execution during continuous activity.

search=debounce; scroll=throttle

Tip: Explain with real UI examples.

Q244. What is lazy loading?

Answer: Loading code, images or data only when needed.

const page=()=>import('./page.js');

Tip: Reduces initial load.

Q245. What is code splitting?

Answer: Breaking a bundle into smaller chunks loaded on demand.

dynamic import()

Tip: Common in modern frameworks.

Q246. What is tree shaking?

Answer: Build tools remove unused exports from statically analyzable modules.

unused named export removed

Tip: ES modules support this well.

Q247. How do you reduce DOM cost?

Answer: Batch updates, use fragments, minimize layout thrashing and delegate events.

const frag=document.createDocumentFragment();

Tip: Measure before optimizing.

Q248. What is XSS?

Answer: Injection of malicious script into a trusted page.

```
unsafe innerHTML with user input
```

Tip: Escape output and avoid unsafe HTML.

Q249. What is prototype pollution?

Answer: Unsafe merging allows attacker-controlled keys to modify object prototypes.

```
payload['__proto__']
```

Tip: Validate keys and use safe libraries.

Q250. What is eval risk?

Answer: eval executes strings as code, creating security, debugging and optimization problems.

```
eval(userInput)
```

Tip: Avoid it.

20. Common Coding and Output Questions

Q251. Reverse a string.

Answer: Convert to an array, reverse it and join it, or loop manually.

```
[... 'hello'].reverse().join('')
```

Tip: Mention Unicode considerations.

Q252. Check a palindrome.

Answer: Normalize the text and compare it with its reverse.

```
const s='level'; s===[...s].reverse().join('');
```

Tip: Clarify case and punctuation rules.

Q253. Remove duplicates from an array.

Answer: Use a Set for primitive values.

```
[...new Set([1,1,2])]
```

Tip: Objects need a key-based strategy.

Q254. Find maximum number.

Answer: Use Math.max with spread or reduce.

```
Math.max(...nums)
```

Tip: Large arrays may prefer reduce.

Q255. Count value frequency.

Answer: Use reduce to build a frequency object or Map.

```
arr.reduce((m,x)=>({m[x]=(m[x]||0)+1,m}),{})
```

Tip: Map supports any key type.

Q256. Flatten nested arrays.

Answer: Use flat with the needed depth or write recursion.

```
arr.flat(Infinity)
```

Tip: Know both built-in and recursive approaches.

Q257. Group objects by a key.

Answer: Reduce into an object or Map keyed by the chosen property.

```
items.reduce((g,x)=>((g[x.type]?=[]).push(x),g),{})
```

Tip: Explain time complexity.

Q258. Find first non-repeating character.

Answer: Count frequencies, then scan in original order.

```
for(const c of s) if(count[c]===1) return c;
```

Tip: Two passes give O(n).

Q259. Implement a simple debounce.

Answer: Clear the previous timer and create a new one for each call.

```
function debounce(fn,d){let t;return(...a)=>{clearTimeout(t);t=setTimeout(()=>fn(...a),d)}}}
```

Tip: Preserve this when needed.

Q260. Implement a simple throttle.

Answer: Track whether execution is allowed and reset after the interval.

```
function throttle(fn,d){let ok=true;return(...a)=>{if(!ok)return;ok=false;fn(...a);setTimeout(()=>ok=true,d)}}}
```

Tip: Variants may preserve trailing calls.

Q261. What is output of typeof null?

Answer: It is 'object' because of a historical language bug.

```
typeof null // 'object'
```

Tip: Use value === null to test null.

Q262. What is output of [] == false?

Answer: true, because loose equality coerces both sides through several conversion steps.

```
[] == false // true
```

Tip: Prefer ===.

Q263. What is output of [] === false?

Answer: false, because the types differ.

```
[] === false // false
```

Tip: Strict equality avoids coercion.

Q264. What is output of 0.1 + 0.2?

Answer: Approximately 0.30000000000000004 due to binary floating-point representation.

```
0.1 + 0.2
```

Tip: Use tolerance or decimal handling.

Q265. What is output order with Promise and timer?

Answer: Synchronous logs run first, then promise microtasks, then timer tasks.

```
console.log(1); Promise.resolve().then(()=>console.log(2)); setTimeout(()=>console.log(3));
```

Tip: Expected order: 1, 2, 3.

Q266. What happens when an object key is another object?

Answer: In a plain object, non-symbol keys are stringified, often causing '[object Object]' collisions.

```
obj[{}]=1;
```

Tip: Use Map for object keys.

21. Scenario and Interview Questions

Q267. API search fires on every keystroke. What do you do?

Answer: Debounce input, cancel stale requests with AbortController, show loading state and ignore out-of-order responses.

```
debounce(loadResults, 300)
```

Tip: Mention UX and network efficiency.

Q268. A page freezes during a large calculation. What do you do?

Answer: Break work into chunks, optimize the algorithm or move CPU-heavy work to a Web Worker.

```
worker.postMessage(data)
```

Tip: Avoid blocking the main thread.

Q269. How do you prevent duplicate form submission?

Answer: Disable the button while submitting, track request state and use idempotency on the server for critical operations.

```
if(submitting)return;
```

Tip: Client protection alone is not enough.

Q270. How do you handle expired authentication?

Answer: Detect 401 responses, refresh the session when supported, retry safely, or redirect to login.

```
interceptor-style fetch wrapper
```

Tip: Avoid infinite refresh loops.

Q271. How do you design reusable API handling?

Answer: Create a wrapper for base URL, headers, JSON parsing, status checks, timeouts and consistent errors.

```
apiRequest('/users', {method: 'GET'})
```

Tip: Keep business logic outside the wrapper.

Q272. How do you debug a memory leak?

Answer: Reproduce it, take heap snapshots, inspect retained objects, check listeners/timers and confirm memory drops after cleanup.

```
Chrome Memory panel
```

Tip: Measure before and after.

Q273. How do you safely render user content?

Answer: Use `textContent`, trusted sanitization and strict output encoding. Avoid assigning untrusted strings to `innerHTML`.

```
element.textContent=input;
```

Tip: Mention XSS.

Q274. How would you explain event delegation in a project?

Answer: One listener on a parent handles many current and future child elements using target matching.

```
table.addEventListener('click', handleRowAction)
```

Tip: Connect it to dynamic dashboards.

Q275. How do you make independent requests faster?

Answer: Start them together and await Promise.all instead of awaiting sequentially.

```
await Promise.all([getUsers(),getStats()])
```

Tip: Only parallelize independent requests.

Q276. How do you prevent stale async results?

Answer: Use cancellation, request IDs or compare the latest query before updating state.

```
if(id!==latestId)return;
```

Tip: Important for typeahead and route changes.

Q277. How do you structure production JavaScript?

Answer: Use modules, clear layers, small functions, linting, tests, centralized error handling and environment configuration.

```
services/, utils/, features/
```

Tip: Discuss maintainability.

Q278. What JavaScript topics are most important for Angular?

Answer: Scope, closures, this, arrays, objects, modules, promises, async/await, event loop and immutability.

Angular uses TypeScript over JavaScript.

Tip: Relate the answer to your stack. Congratulations! You have completed Continue with TypeScript Interview Handbook Revise the examples, explain each answer aloud, and practise coding without copying.